AI Agent 的 Skill 系统设计

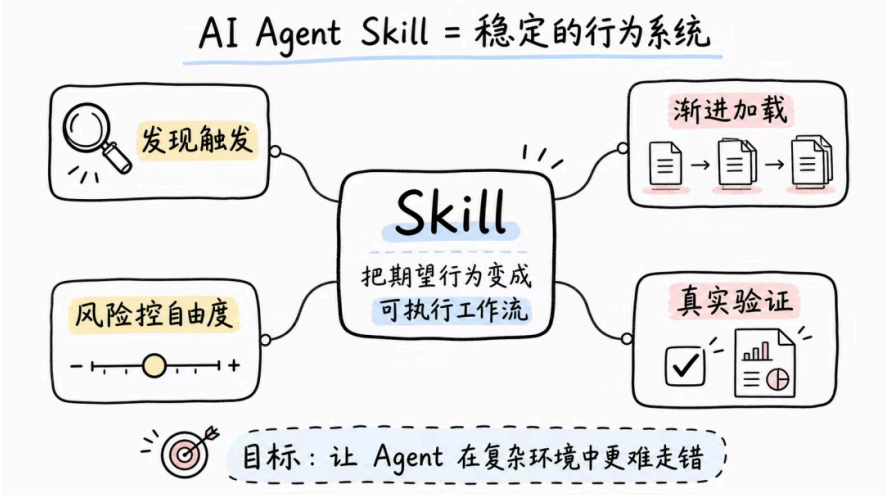
原创 会员技术 大淘宝技术 2026年7月1日 02:27 浙江 97人

本文系统阐述了 AI Agent Skill 系统的设计理念与工程实践，核心观点是将 Skill 视为“行为编程”而非文档，旨在通过结构化设计（YAML+Markdown、DOT 流程图、检查表）和严格的约束机制（门控、合理化防御、说服原则）来规范 AI 代理的行为。文章详细探讨了在有限上下文窗口下的 Token 经济策略，包括基于触发条件的发现机制、两阶段加载及声明式引用；提出了单向管道工作流编排、子代理上下文隔离及分级模型选择方案；强调了基于 TDD 理念的 Skill 测试方法，即通过压力场景观测并封堵代理的违规行为；最后总结了跨平台适配策略及从“建议”走向“强制”、从“手动”走向“自动”的演进教训，旨在构建高合规性、低成本且可维护的 AI 代理技能体系。

Skill 的价值是把期望行为转化成 Agent 能稳定执行的工作流。一个好的 Skill 要同时解决四件事：

1. 让 Agent 在正确场景发现它
2. 用最少上下文加载必要信息
3. 按任务风险设置合适的自由度
4. 并通过真实任务验证它是否改变了行为。

本文的重点是如何写好能被 Agent 正确触发、正确执行、可持续维护的 Skill。



01 先定义目标：Skill 是能力包，不是知识库

Skill 可以被理解为一个自包含的能力包。它通过 SKILL.md、脚本、引用资料和资产，把一个通用 Agent 转化为在特定任务上更可靠的专用 Agent。

它通常提供四类能力：

能力	说明	示例
专用工作流	多步骤、可复用的任务流程	写技术方案、处理 PR 评论、生成报告
工具集成	使用特定文件格式、API 或 CLI 的方法	处理 PDF、调用 GitHub、操作表格
领域知识	业务规则、数据口径、组织约定	公司指标口径、内部权限边界
捆绑资源	脚本、模板、参考资料、素材	scripts/、references/、assets/

Skill 面向的是 Agent，它会在上下文不足、任务复杂、目标冲突或执行压力下走捷径。因此，Skill 必须预设这些失败模式，并把正确路径写成更容易被执行的行为结构。

因此，Skill 不是把人类已经知道的背景知识完整搬进去，而是补充 Agent 完成任务时缺少的程序性知识、资源边界和验证方式。换句话说，写 Skill 不是“把说明写清楚”，而是“让 Agent 在复杂环境中更难走错”。

一个有效的 Skill 要影响 Agent 的完整行为链路：

- 什么时候发现这个 Skill
- 什么时候加载完整正文
- 哪些信息继续按需读取
- 哪些动作必须先做
- 哪些行为绝对不能发生
- 如何证明任务真的完成
- 什么时候需要停下来请求人类判断

如果一个 Skill 只在模型状态好、上下文充足、任务简单时生效，它更像提示词模板。高质量 Skill 应该能在任务复杂、信息不完整、执行压力和合理化冲动下，把 Agent 拉回正确路径。

02 按上下文预算组织内容：元数据、正文、资源三层加载

Skill 设计的第一条工程原则是：上下文窗口是公共资源。

Agent 执行任务时，上下文窗口要同时容纳系统提示、用户请求、对话历史、已触发 Skill、工具结果、代码片段和中间推理。Skill 多占一个 token，其他上下文就少一个 token。所以写 Skill 时要默认 Agent 已经很聪明，只补充它不知道、但完成任务必须知道的内容。

每一段内容都应该经受两个问题的挑战：

- Agent 真的需要这段解释吗？
- 这段内容值得它占用的 token 成本吗？

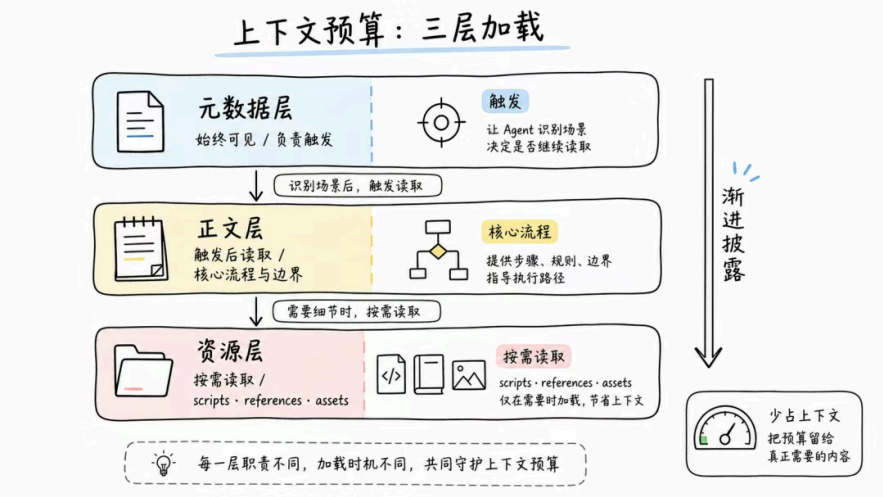
这也是为什么 Skill 应该采用渐进披露，而不是把所有信息塞进一个长文件。

一个标准 Skill 目录通常长这样：

```
skill-name/
SKILL.md
agents/
openai.yaml
scripts/
references/
assets/
```

其中只有 `SKILL.md` 是必需的，其余资源按需要添加。

层级	载入时机	作用	写作原则
元数据层	通常始终可见	让 Agent 判断是否触发	短、准、覆盖触发场景
正文层	Skill 触发后	提供核心流程和边界	保留执行必需内容
资源层	任务需要时	承载细节、脚本、模板	按需读取或执行



SKILL.md 的 frontmatter 和正文要承担不同职责：

```
---
name: create-skill
description: 用于创建或更新包含工作流、工具集成、领域知识、脚本、参考资料或资产的 A
---
# 创建 Skill
流程：
1. 理解具体示例。
2. 规划可复用资源。
3. 初始化 Skill。
4. 编辑 SKILL.md 和相关资源。
5. 验证。
6. 结合真实使用持续迭代。
```

name 和 description 是发现层。正文是执行层。这两层不要混在一起。

description 应该包含“这个 Skill 做什么”和“什么时候使用它”，因为 Agent 只有在触发后才会读取正文。如果把触发条件放在正文里的 When to Use，Agent 在决定是否触发时根本看不到。但 description 也不能变成完整工作流摘要。它的职责是让 Agent 正确加载正文，而不是让 Agent 读完描述就开始凭印象执行。

命名同样属于发现机制的一部分。一个好的 Skill 名称应该短、可触发、动词优先：

好名字	问题名字	原因
`create-skill`	`skill-creation-methodology`	动词入口更像可执行动作
`gh-address-comments`	`github-pr-review-feedback-helper`	命名空间明确，长度更可控
`linear-address-issue`	`issue-processing`	触发对象更清楚

这些细节看起来像命名规范，本质上是路由质量。Agent 在一个不断增长的技能库里找能力，`name` 和 `description` 就是它的第一层索引。



把可复用部分外化：脚本、引用和资产各司其职

创建 Skill 时，不应该一开始就写长篇 `SKILL.md`。更好的路径是先看具体例子，然后判断哪些东西值得变成可复用资源。

资源	适合放什么	不适合放什么
`scripts/`	反复执行、容易写错、需要确定性的操作	只用于解释概念的示意代码
`references/`	schema、API 文档、政策、长示例、领域规则	每次执行都必须读的核心流程
`assets/`	模板、图片、字体、样例工程、PPT 母版	需要被读入上下文的大段说明

当同一段代码会被反复重写，或者任务需要确定性时，放进 `scripts/`：

```
pdf-editor/
SKILL.md
scripts/
rotate_pdf.py
```

脚本的价值不是“让目录更丰富”，而是减少上下文消耗和行为漂移。让 Agent 每次临时生成 PDF 旋转代码，和让它调用一个已经验证过的脚本，是完全不同的可靠性水平。

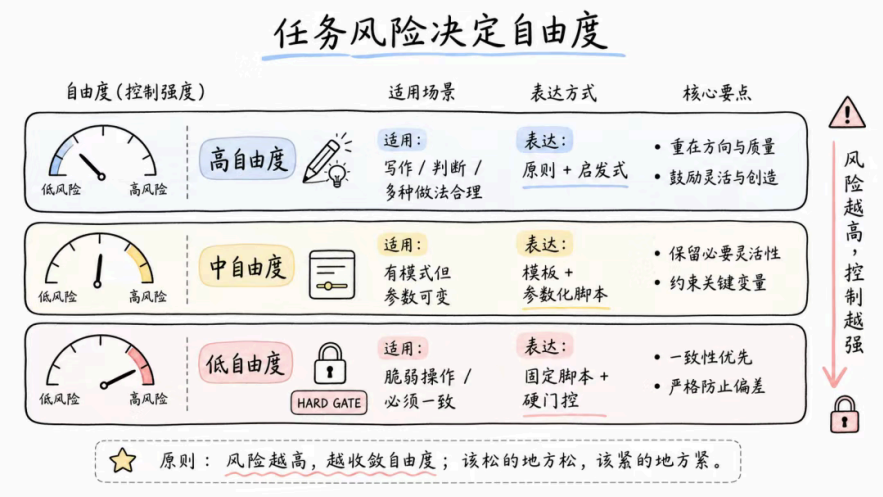
当信息是任务执行时需要查阅的知识，而不是每次都必须读的流程，就放进 `references/`：

```
big-query/
SKILL.md
references/
schema.md
finance.md
product.md
```

如果用户问销售指标，Agent 只需要读 sales.md 或对应领域文件，不应该同时加载财务、产品、市场的所有规则。这就是渐进披露在真实 Skill 中的价值：信息可发现，但不抢占上下文。

当文件不会被读入上下文，而是作为输出材料被复制、修改或引用时，放进 `assets/`：

```
frontend-webapp-builder/
SKILL.md
assets/
hello-world/
```



例如模板工程、字体、图片、PPT 模板、品牌素材都属于资产。它们不是给 Agent 阅读的长文本，而是给最终产物使用的材料。

资源组织还有一个容易被忽略的原则：信息只放一个地方。不要在 `SKILL.md` 和 `references/` 中重复同一段规则。重复会带来漂移，漂移会让 Agent 在两个

版本之间自行解释，最后把维护成本转化成执行风险。

04 按任务风险设置自由度：文本、模板、脚本和门控

Skill 不是越详细越好，也不是越开放越好。关键是让自由度匹配任务的脆弱度和变化空间。

可以把 Skill 的控制方式分成三档：

自由度	适用场景	表达方式
高自由度	多种做法都合理，需要结合上下文判断	文本原则、启发式规则
中自由度	有推荐模式，但允许参数变化	伪代码、参数化脚本、模板
低自由度	操作脆弱、容易出错、必须一致	固定脚本、少量参数、明确顺序

例如：

- 写技术文章：适合高自由度，用结构原则、语气规则和示例引导。
- 查询内部指标：适合中自由度，用 SQL 模板和字段说明控制口径。
- 旋转 PDF、转换格式、生成固定报告：适合低自由度，用脚本保证确定性。

一个常见错误是把脆弱操作写成开放建议，导致 Agent 每次重写一遍容易出错的逻辑。另一个错误是把本该依赖判断的任务写成死流程，导致 Skill 在真实场景里僵硬、不可迁移。

在低自由度任务里，门控尤其重要。Skill 如果只写“建议先做 A”，Agent 很可能直接进入 B。门控的作用是：在条件满足前，明确禁止后续动作。

```
<HARD-GATE>
在理解具体使用示例并规划好可复用资源之前，不要创建或编辑该 Skill。
</HARD-GATE>
```

常见门控包括：

门控类型	用途	示例
前置门控	完成 A 才能开始 B	先理解例子，再设计资源
上下文门控	特定角色或环境跳过流程	子代理不加载入口 Skill
验证门控	有证据才能声明完成	`quick_validate.py` 通过后再交付
审查门控	审查通过才能继续	计划通过后再实现

门控不是语气问题，而是执行边界。它能减少 Agent 的解释空间，让 Skill 在关键路径上更像程序，而不是建议。



把流程写成可执行路径：从例子到 Skill 的创建循环

一个稳妥的 Skill 创建流程可以拆成六步：

1. 理解具体使用例子
2. 规划可复用资源
3. 初始化 Skill
4. 编辑 SKILL.md 和资源
5. 验证 Skill
6. 基于真实使用迭代

这条流程的重点不是“先写一个漂亮的说明”，而是先建立使用边界。

不要从抽象能力开始写 Skill。先问：

- 用户会怎么触发它？
- 哪些请求应该触发？
- 哪些请求不应该触发？
- 任务输入是什么？
- 成功输出是什么？
- 哪些步骤最容易出错？

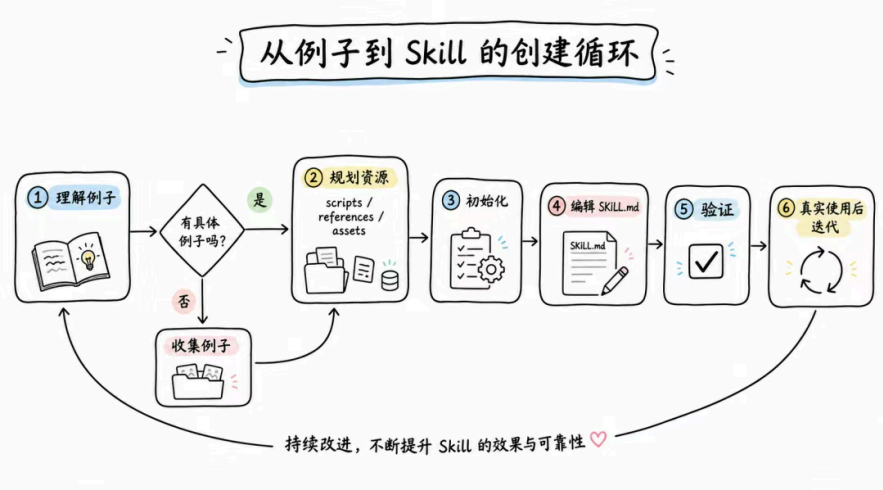
例如做一个 `pdf-editor` Skill，应该先收集“旋转 PDF”“合并 PDF”“提取页面”等具体请求，再决定是否需要脚本。没有具体例子，很容易写出宽泛但不可执行的 Skill。

对每个例子，从零执行一遍，识别可复用部分：

发现	资源选择
同一段代码反复出现	放进 `scripts/`
每次都要查 schema 或规则	放进 `references/`
每次都要复制模板或素材	放进 `assets/`
只是几条核心判断原则	留在 `SKILL.md`

当 Skill 包含非线性判断、循环、回退或容易提前终止的步骤时，流程图比纯文本更稳定。GraphViz DOT 是一个适合嵌入 Markdown 的轻量格式：

```
digraph {
"是否有具体使用例子?" [shape=diamond];
"收集或生成例子" [shape=box];
"规划 scripts/references/assets" [shape=box];
"编写 SKILL.md" [shape=box];
"运行 quick_validate.py" [shape=box];
"完成" [shape=doublecircle];
"是否有具体使用例子?" -> "规划 scripts/references/assets" [label="yes"];
"是否有具体使用例子?" -> "收集或生成例子" [label="no"];
"收集或生成例子" -> "规划 scripts/references/assets";
"规划 scripts/references/assets" -> "编写 SKILL.md";
"编写 SKILL.md" -> "运行 quick_validate.py";
"运行 quick_validate.py" -> "完成";
}
```



复杂流程还需要编号检查表，并要求 Agent 外化进度。否则，Agent 很容易执行前几步后忘记后面的验证和迭代。

初始化 Skill 时，应使用初始化脚本，而不是手写目录结构：

```
scripts/init_skill.py my-skill --path
"${CODEX_HOME:-$HOME/.codex}/skills"
```

如果需要资源目录：

```
scripts/init_skill.py my-skill --path
"${CODEX_HOME:-$HOME/.codex}/skills" --resources scripts,references
```

初始化脚本的意义是减少结构错误，并生成符合规范的模板。之后再替换占位内容，删除不需要的示例文件。



验证不是收尾动作：保护测试完整性，防止合理化

Skill 不应该一次写完就冻结。它的质量来自真实行为反馈，而不是作者对流程的想象。

完成后首先运行基础验证：

```
scripts/quick_validate.py <path/to/skill-folder>
```

基础验证至少应覆盖：

- YAML frontmatter 是否合法
- `name` 和 `description` 是否存在
- 命名是否符合规则
- 资源目录是否合理

- 脚本是否能运行
- UI 元数据是否与 `SKILL.md` 同步

格式验证不能证明 Skill 一定好用，但可以先排除低级错误。复杂 Skill 还需要前向测试。可以用子代理模拟真实用户任务，但要把它当成评估面，而不是审稿人。

正确做法：

```
使用位于 /path/to/skill-x 的 @skill-x 来解决问题 y。
```

不好的做法：

```
审查这个 Skill。我认为它存在问题 A，预期的修复方案是 B。
```

后者会泄露诊断和预期答案，测试结果会被污染。前向测试应该给子代理原始任务、原始材料和最少必要上下文，让它像真实使用者一样执行。

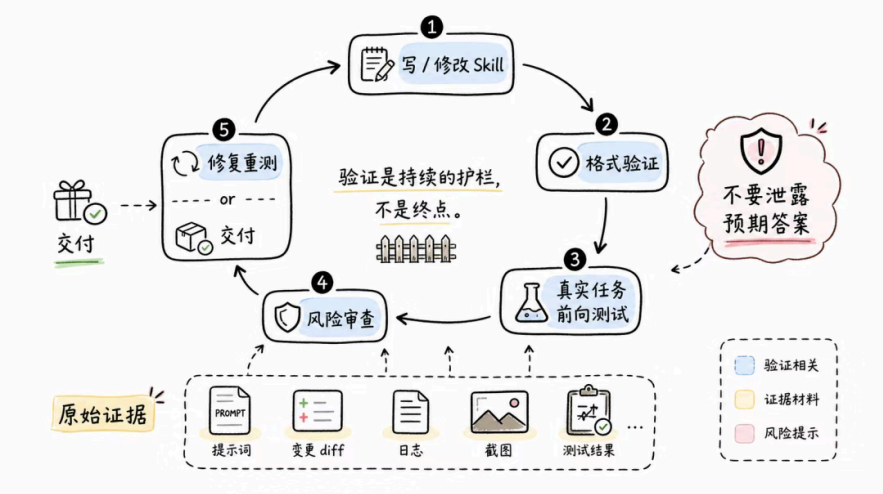
验证时应优先使用原始证据：

- 示例 prompt
- 输出文件
- diff
- 日志
- 行为轨迹
- 失败截图
- 测试结果

如果子代理只有在看到你的结论后才能成功，说明 Skill 本身还不够清楚，或者测试设置已经泄露答案。

这里还要处理一个 Agent 特有的问题：合理化。AI Agent 在压力下会给跳过规则找到听起来合理的理由。Skill 需要提前写出这些借口，并给出反驳。

常见借口	应写进 Skill 的反驳
“这个 Skill 很简单，不需要例子”	没有例子就不知道触发边界，也不知道需要哪些资源。
“先写 SKILL.md，之后再补资源”	资源设计决定正文结构，后补容易造成重复和上下文膨胀。
“脚本很短，每次生成也可以”	重复生成的脆弱代码会漂移，脚本能提供确定性。
“验证只是格式检查，可以跳过”	格式错误会直接影响发现和加载，必须尽早暴露。
“子代理看一下就行”	验证不能泄露预期答案，否则测的是复述能力，不是泛化能力。



审查循环也应该围绕真实失败风险，而不是措辞偏好：

```
写或修改 Skill
-> 运行格式验证
-> 用真实任务前向测试
-> 是否存在会导致任务失败的问题?
-> 是：修复并重测
-> 否：交付
```

应该阻塞的问题包括触发条件模糊、资源引用缺失、脚本不可运行、验证流程缺失、自由度设置错误、关键信息重复且容易漂移。不应该阻塞的问题包括纯粹风格偏好、不影响执行的标题顺序、可以由 Agent 自行判断的轻微表达差异。

当技能库变大后，发现机制会成为第一瓶颈。`description` 是路由器，不是教程。

它应该覆盖：

- Skill 做什么
- 何时使用
- 典型触发词
- 相关症状
- 输入或任务类型

但不要写完整执行流程。

错误写法：

```
description: "创建 Skill 时先收集例子，再规划 scripts/references/assets，然
后运行 init_skill.py，最后 quick_validate.py。"
```

更好的写法：

```
description: "用于创建或更新包含专用工作流、工具集成、领域知识、捆绑脚本、参考
资料、资产、验证或迭代机制的 Agent Skill。"
```

前者让 Agent 可能只凭描述执行，跳过正文。后者让 Agent 知道应该触发，但仍需要加载正文获取完整流程。

Skill 之间也可以互相引用，但应该声明关系，而不是硬编码路径或强制加载大文件：

```
**必需子 Skill：** 创建新 Skill 前，先使用 skill-x。
**推荐：** 如果要将其发布为开发者文章，使用 skill-y。
**另见：** openai_yaml.md，了解 UI 元数据字段。
```

引用可以分为三层：

级别	语义	使用时机
`REQUIRED`	必须先加载	前置依赖
`RECOMMENDED`	建议加载	能明显提高质量
`SEE ALSO`	可选参考	扩展阅读

不要用一次性强制加载大量内容的方式组合 Skill。那会破坏渐进披露，也会让组合 Skill 的成本失控。

最后是平台适配。不同平台的工具名、hook、插件机制和子代理能力可能不同。Skill 应该尽量写行为规则，再用平台层适配具体工具。

例如：

```
- TodoWrite -> todowrite
- Task tool -> @mention subagent system
- Skill tool -> native skill tool
```

平台能力不足时，应优雅降级：

缺失能力	降级策略
无子代理	回退到单会话执行
无启动 hook	用系统提示或入口 Skill 注入
无后台进程	使用前台执行
无 bash	跳过脚本能力，保留文档和资源指导

这能让 Skill 更可迁移，也更容易维护。真正应该稳定的是行为规则，而不是某个平台的私有工具名。

08 交付前自查：反模式和检查表

很多 Skill 失败，不是因为作者不知道领域知识，而是因为它把人类文档的写法带到了 Agent 执行系统里。

常见反模式如下：

反模式	表现	修正
把 Skill 当文档	背景很长，流程很散	只保留可执行知识
`description` 写成教程	Agent 不读正文就开始执行	描述只负责触发
自由度不匹配	脆弱任务只给建议，开放任务写死流程	按任务风险设置自由度
资源不分层	所有内容塞进 `SKILL.md`	用 `scripts/`、`references/`、`assets/` 分离
重复信息	正文和引用文件都有同一规则	信息只放一个地方
跳过验证	写完就交付	运行格式验证和真实任务测试
验证泄露答案	子代理知道预期修复	只给原始任务和材料
额外文档膨胀	创建 README、安装指南、变更日志	只保留执行必需文件

交付前可以用这张表自查：

检查项	问题
具体例子	是否知道用户会怎样触发它?
触发描述	`description` 是否覆盖使用场景、但没有泄露完整流程?
命名	是否短、动词优先、符合 kebab-case?
自由度	是否按任务脆弱度选择文本、伪代码或脚本?
资源分层	是否把脚本、引用、资产放在正确位置?
渐进披露	`SKILL.md` 是否只保留核心流程?
引用清晰	是否明确什么时候读取哪个 reference?
去重	同一信息是否只存在于一个地方?
验证	是否运行了基础验证和必要脚本?
前向测试	复杂 Skill 是否用真实任务测试过?
迭代入口	是否知道真实使用失败后该改哪里?

如果这张表里有多项答不上来，Skill 还不是能力包，只是一份草稿。

09 结论：好 Skill 是小而准的行为系统

写 Skill 不是把最佳实践整理成 Markdown。真正的 Skill 设计要回答三个问题：

1. Agent 在什么情况下应该发现并加载它？
2. Agent 应该获得多少自由度，哪些部分必须被脚本或门控固定？
3. 我们如何用真实任务证明它确实改变了行为？

核心的提醒是：Skill 要简洁、分层、可验证、可迭代。上下文窗口是公共资源，`SKILL.md` 只放核心流程；脚本承接确定性，引用承接领域知识，资产承接输出材料；复杂 Skill 要通过真实任务前向测试，而不是靠作者自信。

因此，Skill 是 Agent 行为设计的一种工程方法。它把触发、加载、执行、约束、验证和迭代组织在一起，让通用 Agent 在特定任务上获得更稳定的专业行为。

10 团队介绍

本文作者苏雄，来自淘天集团–会员技术团队。业务上，我们负责 88VIP、天猫积分、省钱卡、大会员、消费券等淘宝核心业务，同时支撑淘宝、千问、闪购等阿里业务的账号互联互通。技术上，我们深耕 AI 与业务融合，为消费者带来全新体验，为业务创造新增量。

拓展阅读

3DXR技术 | 终端技术 | 音视频技术
服务端技术 | 技术质量 | 数据算法



大淘宝技术
大淘宝技术官方账号
967篇原创内容

公众号

阅读 1.6万

留言

写留言



还没有留言，来写首评